# AIT3002 Midterm Report: QR Code Detection and Decoding

Vũ Nguyên Đan - 23020351

April 20, 2026

## 1 Method Overview

The program reads a CSV listing images, detects every QR code in each image, returns the four-corner quadrilateral of every detection, and optionally decodes the payload. Everything is written from scratch; no QR-specific detection/decoding library (`pyzbar`, `zxing`, `cv2.QRCodeDetector`, etc.) and no deep-learning framework are used. The only third-party dependencies are OpenCV (image I/O, thresholding, contour finding, perspective warp) and NumPy (array math).

The pipeline is:

1. Read the image. If $\max(h, w) > 1024$, resize so the long side is 1024 px (`INTER_AREA`). Work in grayscale; precompute CLAHE on gray (`clipLimit`=3.0, `tileGridSize`=(8,8)) and a light Gaussian blur of that CLAHE output.
2. Run the phased binarization cascade in the next section. Each phase may append a new candidate pool; pools are merged and passed to triplet grouping.
3. Merge near-duplicate finder candidates across pools, cap the merged list, group triplets, estimate corners, deduplicate QRs.
4. Optionally run a 2× upscale pass on small images when few or no QRs are found, and/or a separate dense-grid upscale pass when many small finders are present.
5. Convert each oriented quadrilateral to a 5%-expanded axis-aligned box for `output.csv` `corners`, and keep `oriented_corners` plus `finder_side` for the decoder.
6. If `--decode=yes`, for every detection run the decoder described later in this document.
7. Write `output.csv` with one row per detected QR (or one empty row for images with zero QRs).

## 2 Image preprocessing and detection cascade

All binarizations in Phases 1–5b use the scaled working image. Phase 1 always runs; later phases run only when **no** QR has been assembled yet (`len(qrcodes)==0`) and the total candidate count across pools is $\leq 160$ (circuit breaker). Each binary image feeds `_find_finder_candidates`; if the pool has $\geq 3$ candidates, `_scan_line_verify_np` must pass on at least two of four directions when `strict=True` (Phase 1). Phases 2–5b pass `strict=False` (only one direction required) to recover weak binaries. If verified count falls below three, the unfiltered candidate pool is kept. Each pool is capped at 90 entries.

**Phase 1 (always).**

(a) Otsu on blurred CLAHE gray (`THRESH_BINARY_INV`).

(b) Adaptive Gaussian $15 \times 15$, $C = 2$ on the same blur.

(c) Raw gray adaptive Gaussian $11 \times 11$, $C = 2$ only if small-finder count $\geq 6$ across pools so far, or total finders $< 6$, or average finder area $< 300$ when at least three finders exist.

After Phase 1, `_assemble_qrs` runs on all accumulated pools.

**Phases 2–5b (only if still no QRs and not too many candidates).** In code, these five fallback branches are named Phase 2, Phase 3, Phase 4, Phase 5a, and Phase 5b.

(a) Adaptive Gaussian $11 \times 11$ on CLAHE-equalised gray (no extra blur).

(b) Unsharp mask on CLAHE ($1.8\times$ CLAHE $- 0.8\times$ Gaussian blur $\sigma = 3$), then adaptive Gaussian $15 \times 15$ on a $3 \times 3$ blur of that sharpened image.

(c) Adaptive Gaussian $25 \times 25$ on blurred CLAHE, then morphological close with a $3 \times 3$ kernel; pool kept only if it has $\geq 3$ candidates.

(d) Adaptive mean $21 \times 21$, $C = 4$ on blurred CLAHE; pool kept only if $\geq 3$ candidates.

(e) Raw gray adaptive Gaussian $7 \times 7$, $C = 2$; pool kept only if $\geq 3$ candidates.

**Phase 6 ($2\times$ upscale, independent trigger).** If the original image long side $\leq 720$ px and no QRs are found yet, the original gray image is resized by $2.0\times$. A second trigger is used for small images: if the long side $\leq 640$ px, and $1 \leq |\text{QRs}| \leq 8$, and some detection has `finder_side`$< 22$ px, the original gray image is resized by $2.0\times$. Up to three adaptive Gaussian binarizations (blocks 11, and if many small finders also 9 and 13) run on the upscaled image; new QRs are merged after mapping corners back with `scale`/`up`.

### 2.1 Dense-grid rescue (additional pass)

This pass runs **in addition** to earlier phases when the long side $\leq 1280$ px, there is at least one accumulated pool, and the count of finder candidates with contour area $< 400$ is $\geq 20$. A second trigger: long side $\leq 720$, $1 \leq |\text{QRs}| \leq 10$, and some QR has `finder_side`$< 20$ px. The gray image is upscaled by `up = min(2.0, 1400 / max(h,w))`; the pass is skipped if `up`$< 1.4$. On the upscaled image, adaptive Gaussian (block 11) finder search runs with relaxed scan-line verification ($\geq 1$ direction); pools are capped at 180 candidates. New QRs are merged if not duplicates.

### 2.2 Output geometry

Ground-truth boxes are axis-aligned. Each oriented quadrilateral is converted to its axis-aligned bounding rectangle, then expanded by 5% around the centre (`_BBOX_EXPAND = 1.05`) before integer `corners` are written. `oriented_corners` and `finder_side` are kept for decoding.

## 3 QR Detection

### 3.1 Finder Pattern Candidates

A QR finder pattern is a $7 \times 7$ module square with dark/light/dark/light/dark rings in the ratio 1:1:3:1:1. In a binary image it shows up as three nested contours.

For each binary image the detector:

1. Runs `cv2.findContours` with `RETR_TREE` to obtain the full hierarchy.
2. Looks for chains of three nested contours whose areas roughly satisfy outer/middle $\approx 49/25$ and middle/inner $\approx 25/9$. Tolerances are loose (factor $\leq 12$) because perspective distortion and binarization errors widen the ratios considerably.
3. Rejects the outer contour if $\max(b_w, b_h)/\min(b_w, b_h) > 4.5$ on its bounding rectangle.
4. Verifies each survivor with a scan-line test: four lines through the centroid (horizontal, vertical, two diagonals) are traced, the binary run lengths are compared to the expected 1:1:3:1:1 pattern, and the candidate passes if at least two of four directions match.

Within each direction, any five consecutive runs are tested; each run length $r$ must satisfy $|r - e \cdot u| \leq 0.85\,u + 1$ with unit $u = (\sum r)/7$ and expected pattern $e \in \{1, 1, 3, 1, 1\}$. This blocks most non-finder blobs that still satisfy nested-area ratios.

### 3.2 Candidate Merging

All per-strategy pools are concatenated. In `_merge_candidates`, the parameter `merge_factor` is set to 0.6. The merge radius is

$$r_{ij} = 0.6 \cdot \max\left(\sqrt{\text{area}_i}, \sqrt{\text{area}_j}\right).$$

If Euclidean distance between centres is below this threshold, the two candidates are assigned to the same cluster. Greedy processing in descending `size` keeps one representative per cluster. The merged list is capped at 300 entries before triplet search.

### 3.3 Triplet grouping and assignment

Triplets are enumerated differently by candidate count $n$:

- $n \leq 15$: union-find `_spatial_cluster` splits candidates by distance ($k_factor$ 5.0, then 3.1 on large clusters); all $\binom{|C|}{3}$ triples inside each cluster (plus a "tiny cluster" rescue pool) are tested.
- $n > 15$: for each finder, only pairs drawn from its $K$ nearest neighbours are tried, with $K \in \{4, \ldots, 12\}$ chosen from $n$ (smaller $K$ when $n$ is very large).

`_check_triplet` rejects a triple unless all of the following hold:

- Pairwise $\sqrt{\text{area}}$ ratios between the three finders are all $< 2.3$ (tolerance parameter 1.3).
- Sorted vertex angles: largest angle in $(40°, 140°)$, smallest angle $\geq 8°$.
- From the largest-angle vertex, the two adjacent sides have length ratio $\leq 3.0$.
- Hypotenuse divided by mean adjacent side lies in $(0.85, 2.2)$.
- Each adjacent side length is between $1.6\times$ and $34\times$ the mean $\sqrt{\text{finder area}}$ (lower bound rejects touching finders; upper bound allows high versions with noisy area estimates).
- Timing-pattern sampling along both arms from the right-angle vertex passes heuristic transition-count checks relative to expected module count.
- Estimated version from both area- and bbox-based module sizes lies in a permissive range around $[1, 40]$.
- Arms from the right-angle vertex are not too collinear: $|\cos \angle(\vec{v}_0, \vec{v}_1)| \leq 0.80$.

Score (lower is better) matches the code: largest angle $\theta$ at the right-angle vertex, side-arm ratio $r_{\text{side}}$, hypotenuse ratio $r_{\text{hyp}}$, and normalised size spread $\sigma_{\text{size}}$:

$$s = 2.5\,|\theta - 90°| + 100\,(r_{\text{side}} - 1) + 100\,|r_{\text{hyp}} - 1.414| + 60\,\sigma_{\text{size}}.$$

*Note:* Since $r_{\text{side}} = \max(d_1, d_2)/\min(d_1, d_2) \geq 1$ in this code path, $(r_{\text{side}} - 1)$ is non-negative (equivalent to $|r_{\text{side}} - 1|$), identical to `main.py`.

Post-filters: when $n \geq 20$, triplets whose legs exceed $1.8\times$ the 40th percentile of nearest-neighbour distances are dropped (modal filter against cross-QR triplets). Triplets whose estimated quad contains more than three finder centres in total are always rejected. Finders already used in accepted triplets can be re-grouped in a second pass when $3 \leq \text{unused} \leq 45$.

### 3.4 Corner estimation

From the right-angle vertex and the two opposite finder centres, `_best_module_size` picks a module $m$ consistent with finder-to-finder distances. Outer corners move outward by `offset` $= 4m$ along the two arm directions (not $3.5m$: the extra half-module accounts for label padding). BR blends $0.6\times$ parallelogram completion with $0.4\times$ an extension estimate.

## 4 QR Decoding

Decoding is optional and is controlled by `--decode=yes` or `--decode=no` (default `yes`). When off, decoding is skipped and the `content` field is left empty; when on there is no per-image time budget. `QRDecoder.decode` tries two corner orderings (CW mirror then CCW) and four cyclic rotations of each ($2 \times 4 = 8$ attempts) before giving up on one detection.

### 4.1 Perspective Rectification

The oriented four-corner quadrilateral returned by the detector is warped to a square of side $W = \mathrm{clip}(3 \times \mathrm{side}_{\mathrm{px}}, 240, 960)$ with `cv2.getPerspectiveTransform` and `cv2.warpPerspective`. This first warp is intentionally generous because the detector's quadrilateral is known to overshoot the real QR edge by somewhere between 0 and 1.5 modules.

### 4.2 Finder-Pattern Refinement

Inside the warped image the detector re-runs its finder-pattern search (using three thresholding variants to cope with brightness loss from re-sampling). The three finders closest to the canonical corners $(0, 0)$, $(W, 0)$, $(0, W)$ are selected; from their centres the module size $m$ and version $V$ are recovered with

$$m = \frac{\sqrt{A}}{7}, \qquad V = \frac{N - 17}{4}, \qquad N = \mathrm{round}\big(\tfrac{d_{\mathrm{mean}}}{m}\big) + 7.$$

A second perspective warp maps the three finder centres to their canonical module coordinates (3.5, 3.5), $(N - 3.5,\ 3.5)$, $(3.5,\ N - 3.5)$ at exactly 7 pixels per module, so sampling can be done on a fixed integer grid. If the refinement step fails, the decoder falls back to timing-pattern version estimation combined with a brute-force search over margin values (in order) $\{1.0,\ 0.5,\ 1.5,\ 0.0\}$ modules.

### 4.3 Bit matrix sampling

After finder refinement, each module is $7 \times 7$ pixels; the bit value uses the centre pixel of that block compared to a threshold map from `_adaptive_bit_threshold`: global Otsu on the sample grid (clamped to $[30, 225]$), plus a box blur whose kernel size is $\max(3, \lfloor N/4 \rfloor)$ (odd), blended as $0.7 \times \mathrm{local} + 0.3 \times \mathrm{global}$. On the pre-refinement fallback path, `cv2.remap` samples the first warp on a regular grid; quiet-zone margin is tried in order $\{1.0, 0.5, 1.5, 0.0\}$ modules.

Before committing to a sampled matrix the decoder checks that at least two of the three finder-corner $7 \times 7$ blocks actually look like a finder (dark outer ring, dark inner $3 \times 3$). This catches misaligned samplings that would otherwise decode into nonsense.

### 4.4 Format Information

The 15-bit format string is read from two redundant locations on the module grid. After XOR with the fixed mask `0x5412` the 32 valid code words are compared by Hamming distance; the closest is kept if its distance is $\leq 3$. The decoded bits give the error-correction level ($\{L, M, Q, H\}$) and mask index (0−7).

### 4.5 Zig-Zag Extraction and Unmasking

Data is extracted with the standard zig-zag walker: columns are traversed in pairs right-to-left, alternating bottom-up and top-down; column 6 (the vertical timing column) is skipped; modules flagged as function (finders, separators, timing, format, alignment, version info, dark module) are skipped. For each data module, the selected mask function is XOR-ed with the bit value.

### 4.6 Reed–Solomon Error Correction

Reed–Solomon arithmetic is performed in $GF(2^8)$ using the QR-standard primitive polynomial $x^8 + x^4 + x^3 + x^2 + 1$ (hex `0x11D`). The implementation comprises:

- Precomputed log / anti-log tables for $GF(256)$ arithmetic.
- Syndrome evaluation.
- **Berlekamp–Massey** for the error-locator polynomial.
- Chien search for error positions.
- **Forney algorithm** for error magnitudes.

For versions with multiple blocks, codewords are first de-interleaved into their blocks according to the $(c, d, e)$ triples from the ISO 18004 Appendix-I tables, each block is corrected independently, and the data portions are concatenated.

### 4.7 Data Decoding

The bit stream is parsed by four-bit mode indicators:

| Mode | Indicator | Packing |
|---|---|---|
| Numeric | `0001` | 3 digits / 10 bits, 2 / 7, 1 / 4 |
| Alphanumeric | `0010` | pair / 11 bits, single / 6 |
| Byte | `0100` | 8 bits/byte, decoded as UTF-8 with Latin-1 / Shift-JIS fallback |
| Kanji | `1000` | 13 bits/char, Shift-JIS |

ECI designators, FNC1 markers and structured-append headers are skipped rather than implemented, which is sufficient for the vast majority of QRs in the data set.

## 5 Libraries and Algorithms

Third-party dependencies:

- **OpenCV $\geq$ 4.5** – `imread`, `cvtColor`, `createCLAHE`, `adaptiveThreshold`, `threshold` (`OTSU`), `findContours`, `boundingRect`, `morphologyEx`, `getPerspectiveTransform`, `warpPerspective`, `remap`, `resize`. No `QRCodeDetector`, `detectAndDecode`, or related QR-specific APIs.
- **NumPy $\geq$ 1.20** – numerical arrays.
- Standard library: `argparse`, `csv`, `multiprocessing`, `concurrent.futures`, etc.

`requirements.txt` lists only OpenCV and NumPy. The header comment in `main.py` mentions SciPy; no SciPy import is used at runtime.

Core algorithms, all implemented from scratch: contour-hierarchy finder detection, scan-line 1:1:3:1:1 verification, triplet geometric scoring, adaptive $k$-nearest-neighbour triplet enumeration for large $n$, spatial union–find clustering for small $n$, dense-grid modal and phantom triplet filters, dense-QR upscale rescue, perspective rectification, adaptive Otsu, BCH(15, 5) format-code

decoding, GF($2^8$) arithmetic, Berlekamp–Massey, Chien search, Forney, zig-zag data extraction, four-mode data parser.

No machine-learning model is trained or used.

# 6 Results on the Public Sets

Runs were executed with 12 worker processes on a consumer CPU. Each measurement below is wall-clock time reported by the program itself and includes process-pool startup. Table values are from one run per setting; wall-clock can vary across runs.

## 6.1 Detection Only (`--decode=no`)

| Split | Images | GT QRs | TP | FP | FN | F1 | Time |
|---|---|---|---|---|---|---|---|
| train | 1083 | 1979 | 1184 | 154 | 795 | **0.7139** | 15.2 s (14 ms/img) |
| valid | 309 | 508 | 359 | 36 | 149 | **0.7951** | 6.2 s (20 ms/img) |

Precision is 0.885 (train) and 0.909 (valid); recall is 0.598 / 0.707. In the measured run, wall-clock time is about 14 ms/image on the train split and about 20 ms/image on the valid split under the measured hardware (valid has fewer images so fixed pool overhead is amortised less).

## 6.2 Detection + Decoding (`--decode=yes`)

F1 is unchanged from above (decoding does not alter the predicted quadrilaterals). The table adds decoding statistics.

| Split | Detected QRs | Decoded | Decode rate | Time |
|---|---|---|---|---|
| train | 1338 | 280 | 20.9 % | 39.9 s (37 ms/img) |
| valid | 395 | 89 | 22.5 % | 17.3 s (56 ms/img) |

The provided ground-truth CSVs leave the `content` column empty, so public-split content correctness cannot be measured. In `evaluate.py`, `Content_Accuracy` is computed over total TP, which follows `output_requirement.md` section 5.4; with empty GT content on public, this value is not informative for decoder correctness. Inspection of the decoded strings shows that whenever the decoder succeeds, the output is meaningful (URLs, VCards, `TEL:`, `MAILTO:`, and the test strings "`Version 1 QR`" / "`Version 2 QR Code Test Image`" that appear in several categories).

## 6.3 Representative Outputs

Successful decodes include:

- `http://www.sneakyparties.com.au`
- `http://qr.westergasfabriek.nl/480`
- `BEGIN:VCARD\n VERSION:2.1 \n N:Gump;Forrest ...`
- `ANA080514DHGE38V4TBT246927890 2070031`
- `TEL:+4992815939305`

Typical detection failures:

- Heavy motion blur — the scan-line test stops validating the 1:1:3:1:1 pattern even when all three finders are visually present.
- Extreme perspective where two finders almost overlap; the right-angle test fails.

- Partially occluded QRs where one of the three finders is physically missing (the pipeline requires three).
- Very small QRs in `lotsimage` (module size below $\sim 1.5$ px after downscaling) even after the upscale rescue.

Typical decoding failures (given that detection succeeded):

- Finder-pattern refinement picks a spurious finder (text or a reflection mimicking one), which throws the sampling grid off by a full module.
- High noise + curved surfaces: the format code survives but the data portion has too many bit errors for Reed–Solomon to correct.
- Version $\geq 7$: version info block is not used for cross-check, so if timing-based version estimation is wrong by one, the grid is mis-sampled.

### 6.4 Speed

Detection runtime in the measured public runs is 14 ms/image (train) and 20 ms/image (valid) without decoding. Decoding is slower because each detection tries eight corner orderings, then finder refinement; if that fails, up to four versions are tried and for each version up to four quiet-zone margins are tested before giving up. Up to twelve (EC level, mask) pairs are tried per matrix. Decoding can be disabled with `--decode=no`.

### 6.5 Reproducibility

The pipeline is deterministic: no random sampling and no randomized seed. `ProcessPoolExecutor.map` preserves input order, so for fixed code and input CSV order, output row ordering is deterministic.

## 7 Analysis and Discussion

### 7.1 Difficulties

- Dense grid images (often filenames containing `lotsimage`) produce large finder candidate sets. Without spatial clustering plus nearest-neighbour triplet pruning, exhaustive triplet search would be too slow; without the modal and phantom filters, many cross-QR false triplets would survive.
- The detector's quadrilateral does not have a fixed margin around the real QR edge; it varies between 0 and $\sim 1.5$ modules depending on which rescue path produced it. Decoding only became reliable after the second, finder-anchored warp was added.
- Without ground-truth content strings there is no automatic signal for decoder regressions; only the detection F1 and the count of non-empty content rows were available during development.

### 7.2 Limitations

- Requires all three finder patterns: any QR with one finder fully occluded is missed.
- The format/mask search is a brute-force fallback when BCH decoding fails; it can occasionally confirm a wrong mask because a few mask+EC combinations happen to yield plausible mode indicators.
- Version $\geq 7$ version-info codewords are not decoded, so an off-by-one version estimate on large QRs goes uncaught.

### 7.3 Possible Improvements

- Two-finder rescue: when two clean finders are found but the third is missing, synthesize a candidate from timing-pattern extrapolation.
- Use detected alignment-pattern centres to refine the perspective warp for versions $\geq 2$, reducing the misalignment that currently dominates the decoding failure cases.
- Decode version info for $V \geq 7$ to eliminate the timing-based off-by-one.
- Add a lightweight quality gate on the decoded text (UTF-8 validity, printable ratio) before accepting it, to suppress the rare but present false positives from brute-force mask search.
- Port the critical inner loops of the scan-line verifier and zig-zag walker to Cython or Numba to drop detection time further.

## 8 Overfitting and Generalization

The system is a hand-written rule-based pipeline; no model parameters are learned from the public training set. All thresholds (finder-size limits, triplet-scoring weights, morphological kernel sizes, adaptive block sizes, etc.) were chosen from first principles or from the QR specification. The numbers that depend on the data are:

1. Numeric constants in the cascade (pool caps 90/300/180, merge factor 0.6, circuit breaker 160, dense triggers with small-finder area threshold 400, upscale limits, modal-filter percentile 40 and multiplier 1.8, etc.).
2. The 5% axis-aligned box expansion (`_BBOX_EXPAND = 1.05`) chosen from a sweep on the public training labels.
3. Scan-line ratio tolerance (default 0.85 in `_check_ratio_runs`) and triplet scoring weights.

These are hand-set constants, not weights from gradient-based training. Behaviour on a private test set can differ if image statistics differ, but nothing in the pipeline memorises per-image labels from the public CSVs.

Re-running the evaluation on the train or valid split gives identical `output.csv` files (no randomness, no timing-dependent behaviour), so any cross-run variation in reported F1 will come from changes to the program itself, not from stochasticity.

## 9 How to Run

```
pip install -r requirements.txt
python main.py --data qr/public_train.csv # default: decode=yes
python main.py --data qr/public_valid.csv --decode=no
python evaluate.py qr/output_valid.csv output.csv
```

The program always writes `output.csv` next to `main.py`. The workers count is auto-capped at $\min(12, \text{cpu_count})$.